

SAPIENZA
UNIVERSITÀ DI ROMA

Algorithms

Exercises 2 — Programming languages

Worked exercises on chapters 6–8: programs in C++, Python and Java, from pseudo-code to code. Every exercise has a complete solution.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/algorithms/
github.com/fabiofurini/algorithms

Academic year 2026–2027

Contents

6 Introduction to C++ **2**

7 Introduction to Python **26**

8 Introduction to Java **40**

Chapter 6

Introduction to C++

In the exercises of this chapter the programs are written, compiled and executed with the conventions of the chapter «Introduction to C++», summarized below.

Warning

- The program of an exercise is saved in a file `<name>.cpp`, compiled and executed with the commands

```
g++ -Wall -std=c++17 -o <name> <name>.cpp
./<name>
```

- The test instances are fixed-size arrays declared in `main`, for example `int arr1[] = {9, 12, 3, 16, 19, 11, 12};`, and are passed to the functions through the pointer to their first element (`int *a`) together with their size `n`.
- The elements of an array of size n are `a[0]`, `a[1]`, ..., `a[n-1]`: the entry $a[i]$ of an instance written as $a = [\dots]$ in the text corresponds to `a[i-1]` in the code.
- The results that a function has to give back besides its return value are passed by reference (`int& x`, `double& x`).
- When an array is printed, every value is followed by a space, so the printed lines end with a space.

Exercise 6.1 — Introduction to C++ ($E1$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of finding the maximum value in the array.

Questions

1. Write a complete C++ program that finds the maximum value in an array a of n integer numbers. The program should:

- Define a function `int ArrayMaxValue(int n, int *a)` that implements the algorithm
- In the `main()` function, create test arrays and call the function
- Print the results for each test case

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the complete compilation commands and output for all test cases.

3. Modify the program to create a new function `int ArrayMaxValueWithIndex(int n, int *a, int& idx)` that returns both the maximum value and stores its index in the reference parameter `idx`. Execute this modified version and show the output.

Solution

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Function to find maximum value in array
int ArrayMaxValue(int n, int *a)
{
    int max = a[0];  // Initialize with first element

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];  // Update maximum
        }
    }

    return max;
}

int main()
{
    // Test case 1
    int arr1[] = {9, 12, 3, 16, 19, 11, 12};
    int n1 = 7;

    cout << "Test 1 - Array: ";
    for (int i = 0; i < n1; i++)
    {
```

```
        cout << arr1[i] << " ";
    }
    cout << endl;

    int max1 = ArrayMaxValue(n1, arr1);
    cout << "Maximum value: " << max1 << endl;
    cout << endl;

    // Test case 2
    int arr2[] = {5, 2, 8, 1};
    int n2 = 4;

    cout << "Test 2 - Array: ";
    for (int i = 0; i < n2; i++)
    {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int max2 = ArrayMaxValue(n2, arr2);
    cout << "Maximum value: " << max2 << endl;
    cout << endl;

    // Test case 3
    int arr3[] = {42};
    int n3 = 1;

    cout << "Test 3 - Array: ";
    for (int i = 0; i < n3; i++)
    {
        cout << arr3[i] << " ";
    }
    cout << endl;

    int max3 = ArrayMaxValue(n3, arr3);
    cout << "Maximum value: " << max3 << endl;

    return 0;
}
```

2. **Compilation and execution:**

   To compile the program:

   ```
   g++ -Wall -std=c++17 -o max_value max_value.cpp
   ```

   To run the program:

   ```
   ./max_value
   ```

   Output:

```
Test 1 - Array: 9 12 3 16 19 11 12
Maximum value: 19

Test 2 - Array: 5 2 8 1
Maximum value: 8

Test 3 - Array: 42
Maximum value: 42
```

3. **Modified program with index:**

```cpp
#include <iostream>
using namespace std;

// Function to find maximum value and its index
int ArrayMaxValueWithIndex(int n, int *a, int& idx)
{
    int max = a[0];
    idx = 0;

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];
            idx = i;  // Store the index
        }
    }

    return max;
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;
    int index;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    int max = ArrayMaxValueWithIndex(n, arr, index);

    cout << "Maximum value: " << max << endl;
    cout << "Position (index): " << index << endl;

    return 0;
}
```

Output:

```
Array: 9 12 3 16 19 11 12
Maximum value: 19
Position (index): 4
```

Note: The maximum value 19 is found at index 4 (the 5th element, since arrays are indexed from 0 in C++). If the maximum value appears more than once, the function stores the index of its first occurrence, because `max` and `idx` are updated only when `a[i] > max` (strict inequality).

Exercise 6.2 — Introduction to C++ ($E2$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of finding its minimum value.

Questions

1. Write a complete C++ program that finds the minimum value in an array a of n integer numbers. The program should:
   - Define a function `int ArrayMinValue(int n, int *a)` that implements the algorithm
   - In the `main()` function, create test arrays and call the function
   - Print the results for each test case

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [15, 7, 22, 4]$ with $n = 4$
   - Instance 3: $a = [100]$ with $n = 1$

   Show the complete compilation commands and output for all test cases.

3. Modify the program to create a new function `void ArrayMaxMin(int n, int *a, int& max, int& min)` that finds both the maximum and minimum values in a single pass through the array. Execute this modified version and show the output.

Solution

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Function to find minimum value in array
int ArrayMinValue(int n, int *a)
{
    int m = a[0];  // Initialize with first element

    for (int i = 1; i < n; i++)
    {
        if (a[i] < m)
        {
            m = a[i];  // Update minimum
        }
    }

    return m;
}

int main()
{
    // Test case 1
    int arr1[] = {9, 12, 3, 16, 19, 11, 12};
    int n1 = 7;

    cout << "Test 1 - Array: ";
    for (int i = 0; i < n1; i++)
    {
        cout << arr1[i] << " ";
    }
    cout << endl;

    int min1 = ArrayMinValue(n1, arr1);
    cout << "Minimum value: " << min1 << endl;
    cout << endl;

    // Test case 2
    int arr2[] = {15, 7, 22, 4};
    int n2 = 4;

    cout << "Test 2 - Array: ";
    for (int i = 0; i < n2; i++)
    {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int min2 = ArrayMinValue(n2, arr2);
    cout << "Minimum value: " << min2 << endl;
    cout << endl;

    // Test case 3
    int arr3[] = {100};
    int n3 = 1;

    cout << "Test 3 - Array: ";
```

```
    for (int i = 0; i < n3; i++)
    {
        cout << arr3[i] << " ";
    }
    cout << endl;

    int min3 = ArrayMinValue(n3, arr3);
    cout << "Minimum value: " << min3 << endl;

    return 0;
}
```

2. **Compilation and execution:**

   To compile the program:

   ```
   g++ -Wall -std=c++17 -o min_value min_value.cpp
   ```

   To run the program:

   ```
   ./min_value
   ```

   Output:

   ```
   Test 1 - Array: 9 12 3 16 19 11 12
   Minimum value: 3

   Test 2 - Array: 15 7 22 4
   Minimum value: 4

   Test 3 - Array: 100
   Minimum value: 100
   ```

3. **Modified program to find both max and min:**

```
#include <iostream>
using namespace std;

// Function to find both maximum and minimum values
void ArrayMaxMin(int n, int *a, int& max, int& min)
{
    max = a[0];
    min = a[0];

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];
```

```
        }
        if (a[i] < min)
        {
            min = a[i];
        }
    }
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;
    int maximum, minimum;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    ArrayMaxMin(n, arr, maximum, minimum);

    cout << "Maximum value: " << maximum << endl;
    cout << "Minimum value: " << minimum << endl;

    return 0;
}
```

Output:

```
Array: 9 12 3 16 19 11 12
Maximum value: 19
Minimum value: 3
```

Note: This implementation finds both the maximum and the minimum value in a single pass through the array. The number of comparisons is the same as with two separate functions, $2(n-1)$, but the array is traversed once instead of twice.

Exercise 6.3 — Introduction to C++ ($E3$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of computing the sum and average of all its elements.

Questions

1. Write a complete C++ program that computes the sum and average of elements in an array a of n integer numbers. The program should:

- Define a function `int ArraySum(int n, int *a)` that computes the sum of array elements
- In the `main()` function, create test arrays and call the function
- Compute and print both the sum and average for each test case

2. Execute the program on the following instances:
   - Instance 1: $a = [5, 12, 3, 8]$ with $n = 4$
   - Instance 2: $a = [10, 20, 30]$ with $n = 3$
   - Instance 3: $a = [7]$ with $n = 1$

   Show the complete compilation commands and output for all test cases.

3. Modify the program to create a new function `void ArraySumAndAverage(int n, int *a, int& sum, double& avg)` that computes both the sum and average in one function. Execute this modified version and show the output.

Solution

1. **Complete C++ program:**

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// Function to compute sum of array elements
int ArraySum(int n, int *a)
{
    int S = 0;

    for (int i = 0; i < n; i++)
    {
        S = S + a[i];
    }

    return S;
}

int main()
{
    // Test case 1
    int arr1[] = {5, 12, 3, 8};
    int n1 = 4;

    cout << "Test 1 - Array: ";
    for (int i = 0; i < n1; i++)
    {
        cout << arr1[i] << " ";
    }
```

```
    cout << endl;

    int sum1 = ArraySum(n1, arr1);
    double avg1 = (double)sum1 / n1;
    cout << "Sum: " << sum1 << endl;
    cout << "Average: " << fixed << setprecision(2)
         << avg1 << endl;
    cout << endl;

    // Test case 2
    int arr2[] = {10, 20, 30};
    int n2 = 3;

    cout << "Test 2 - Array: ";
    for (int i = 0; i < n2; i++)
    {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int sum2 = ArraySum(n2, arr2);
    double avg2 = (double)sum2 / n2;
    cout << "Sum: " << sum2 << endl;
    cout << "Average: " << fixed << setprecision(2)
         << avg2 << endl;
    cout << endl;

    // Test case 3
    int arr3[] = {7};
    int n3 = 1;

    cout << "Test 3 - Array: ";
    for (int i = 0; i < n3; i++)
    {
        cout << arr3[i] << " ";
    }
    cout << endl;

    int sum3 = ArraySum(n3, arr3);
    double avg3 = (double)sum3 / n3;
    cout << "Sum: " << sum3 << endl;
    cout << "Average: " << fixed << setprecision(2)
         << avg3 << endl;

    return 0;
}
```

2. **Compilation and execution:**

   To compile the program:

   ```
   g++ -Wall -std=c++17 -o array_sum array_sum.cpp
   ```

   To run the program:

```
./array_sum
```

Output:

```
Test 1 - Array: 5 12 3 8
Sum: 28
Average: 7.00

Test 2 - Array: 10 20 30
Sum: 60
Average: 20.00

Test 3 - Array: 7
Sum: 7
Average: 7.00
```

3. **Modified program with combined function:**

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// Function to compute both sum and average
void ArraySumAndAverage(int n, int *a, int& sum, double& avg)
{
    sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum = sum + a[i];
    }

    avg = (double)sum / n;
}

int main()
{
    int arr[] = {5, 12, 3, 8};
    int n = 4;
    int sum;
    double average;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    ArraySumAndAverage(n, arr, sum, average);

```

```
34      cout << "Sum: " << sum << endl;
35      cout << "Average: " << fixed << setprecision(2)
36           << average << endl;
37
38      return 0;
39  }
```

Output:

```
Array: 5 12 3 8
Sum: 28
Average: 7.00
```

Note: The `setprecision(2)` manipulator is used to format the average with 2 decimal places. The conversion `(double)` ensures the floating-point division: without it, `sum / n` would be the integer division. Since the average divides by n, the functions require $n \geq 1$.

Exercise 6.4 — Introduction to C++ ($E4$)

Given an array a of n integer numbers and a value m, we consider the problem of determining whether the value m is present or not in the array.

Questions

1. Write a complete C++ program that implements the linear search algorithm for an array a of n integer numbers. The program should:
   - Define a function `int LinearSearch(int n, int *a, int m)` that searches for value `m` in array `a`
   - Return the index of the element if found, or `-1` if not found
   - In the `main()` function, create a test array and search for different values
   - Print the results for each search

2. Execute the program with the following instance:
   - Array: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Search for: $m = 16$ (element present)
   - Search for: $m = 25$ (element not present)
   - Search for: $m = 9$ (first element)
   - Search for: $m = 12$ (duplicate element)

Show the complete compilation commands and output for all test cases.

3. Modify the program to create a function `int CountOccurrences(int n, int *a, int m)` that counts how many times value `m` appears in the array. Execute this modified version with value $m = 12$ in the test array and show the output.

Solution

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Function to perform linear search
int LinearSearch(int n, int *a, int m)
{
    for (int i = 0; i < n; i++)
    {
        if (a[i] == m)
        {
            return i;  // Found, return index
        }
    }

    return -1;  // Not found
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl << endl;

    // Test 1: Search for 16 (present)
    int m1 = 16;
    int result1 = LinearSearch(n, arr, m1);
    cout << "Search for " << m1 << ": ";
    if (result1 != -1)
    {
        cout << "Found at index " << result1 << endl;
    }
    else
    {
        cout << "Not found" << endl;
    }

    // Test 2: Search for 25 (not present)
```

```
    int m2 = 25;
    int result2 = LinearSearch(n, arr, m2);
    cout << "Search for " << m2 << ": ";
    if (result2 != -1)
    {
        cout << "Found at index " << result2 << endl;
    }
    else
    {
        cout << "Not found" << endl;
    }

    // Test 3: Search for 9 (first element)
    int m3 = 9;
    int result3 = LinearSearch(n, arr, m3);
    cout << "Search for " << m3 << ": ";
    if (result3 != -1)
    {
        cout << "Found at index " << result3 << endl;
    }
    else
    {
        cout << "Not found" << endl;
    }

    // Test 4: Search for 12 (duplicate)
    int m4 = 12;
    int result4 = LinearSearch(n, arr, m4);
    cout << "Search for " << m4 << ": ";
    if (result4 != -1)
    {
        cout << "Found at index " << result4
             << " (first occurrence)" << endl;
    }
    else
    {
        cout << "Not found" << endl;
    }

    return 0;
}
```

2. **Compilation and execution:**

   To compile the program:

   ```
   g++ -Wall -std=c++17 -o linear_search linear_search.cpp
   ```

   To run the program:

   ```
   ./linear_search
   ```

   Output:

```
Array: 9 12 3 16 19 11 12

Search for 16: Found at index 3
Search for 25: Not found
Search for 9: Found at index 0
Search for 12: Found at index 1 (first occurrence)
```

3. **Modified program to count occurrences:**

```cpp
#include <iostream>
using namespace std;

// Function to count occurrences of value m
int CountOccurrences(int n, int *a, int m)
{
    int count = 0;

    for (int i = 0; i < n; i++)
    {
        if (a[i] == m)
        {
            count++;
        }
    }

    return count;
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    int x = 12;
    int count = CountOccurrences(n, arr, x);

    cout << "Value " << x << " appears " << count
         << " times in the array" << endl;

    return 0;
}
```

Output:

```
Array: 9 12 3 16 19 11 12
Value 12 appears 2 times in the array
```

Note: The `LinearSearch` function returns the index of the first occurrence when duplicates exist. The `CountOccurrences` function scans the entire array to count all occurrences.

Exercise 6.5 — Introduction to C++ ($E5$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of computing in a single pass the maximum, minimum, sum and average of all elements in the array.

Questions

1. Write a complete `C++` program that computes all statistics for an array a of n integer numbers in a single pass. The program should:
   - Define a function `void ArrayStatistics(int n, int *a, int& max, int& min, int& sum, double& avg)` that computes all four statistics
   - In the `main()` function, create test arrays and call the function
   - Print all statistics for each test case

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$

   Show the complete compilation commands and output for all test cases.

3. Discuss the advantages of computing all statistics in a single pass versus computing them separately with multiple functions.

Solution

1. **Complete `C++` program:**

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// Function to compute all statistics in a single pass
void ArrayStatistics(int n, int *a, int& max, int& min,
                     int& sum, double& avg)
{
    // Initialize with first element
    max = a[0];
    min = a[0];
    sum = a[0];
```

```
13
14      // Process remaining elements
15      for (int i = 1; i < n; i++)
16      {
17          if (a[i] > max)
18          {
19              max = a[i];
20          }
21          if (a[i] < min)
22          {
23              min = a[i];
24          }
25          sum = sum + a[i];
26      }
27
28      // Compute average
29      avg = (double)sum / n;
30  }
31
32  int main()
33  {
34      // Test case 1
35      int arr1[] = {9, 12, 3, 16, 19, 11, 12};
36      int n1 = 7;
37      int max1, min1, sum1;
38      double avg1;
39
40      cout << "Test 1 - Array: ";
41      for (int i = 0; i < n1; i++)
42      {
43          cout << arr1[i] << " ";
44      }
45      cout << endl;
46
47      ArrayStatistics(n1, arr1, max1, min1, sum1, avg1);
48
49      cout << "Maximum: " << max1 << endl;
50      cout << "Minimum: " << min1 << endl;
51      cout << "Sum: " << sum1 << endl;
52      cout << "Average: " << fixed << setprecision(2)
53           << avg1 << endl;
54      cout << endl;
55
56      // Test case 2
57      int arr2[] = {5, 2, 8, 1};
58      int n2 = 4;
59      int max2, min2, sum2;
60      double avg2;
61
62      cout << "Test 2 - Array: ";
63      for (int i = 0; i < n2; i++)
64      {
65          cout << arr2[i] << " ";
66      }
67      cout << endl;
68
```

```
    ArrayStatistics(n2, arr2, max2, min2, sum2, avg2);

    cout << "Maximum: " << max2 << endl;
    cout << "Minimum: " << min2 << endl;
    cout << "Sum: " << sum2 << endl;
    cout << "Average: " << fixed << setprecision(2)
         << avg2 << endl;

    return 0;
}
```

2. **Compilation and execution:**

   To compile the program:

   ```
   g++ -Wall -std=c++17 -o array_stats array_stats.cpp
   ```

   To run the program:

   ```
   ./array_stats
   ```

   Output:

   ```
   Test 1 - Array: 9 12 3 16 19 11 12
   Maximum: 19
   Minimum: 3
   Sum: 82
   Average: 11.71

   Test 2 - Array: 5 2 8 1
   Maximum: 8
   Minimum: 1
   Sum: 16
   Average: 4.00
   ```

3. **Advantages of single-pass approach:**

   Advantages:

   - **Fewer array traversals:** The array is traversed only once instead of once for each statistic
   - **More compact code:** Single function and single loop instead of multiple function calls
   - **Better for streaming data:** Can process data that arrives sequentially or doesn't fit entirely in memory

- **Reduced overhead:** Only one loop initialization and termination instead of multiple loops

Trade-offs:

- **Less modular:** Combines multiple operations in one function, which may be harder to maintain
- **Less flexible:** If only one statistic is needed, the function still computes all of them
- **Code reusability:** Cannot easily reuse individual statistic computations in other contexts

Recommendation: The single-pass approach is preferable when:

- All statistics are needed together
- Working with large arrays where minimizing array accesses is important
- Processing streaming data or data that cannot fit in memory

Separate functions are better when:

- Only specific statistics are needed
- Code clarity and modularity are higher priorities
- Individual functions may be reused in different contexts

Exercise 6.6 — Introduction to C++ ($E6$)

Given a two-dimensional array a of $m \times n$ integer numbers, we consider the problem of computing the sum of all its elements.

Questions

1. Write a complete C++ program that computes the sum of all elements of a two-dimensional array a of $m \times n$ integers. The program should:
   - Define a function `int SumMatrix(int m, int n, int **matrix)` that computes the sum
   - In the `main()` function, allocate test matrices using dynamic memory, initialize them, call the function, and print the results
   - Free all allocated memory after use

2. Execute the program on the following instances:

- Instance 1: a 3×4 matrix with sequential values $a[i][j] = (i-1) \cdot n + j$ for $i \in \{1,2,3\}$ and $j \in \{1,2,3,4\}$ (rows and columns numbered from 1, as in pseudo-code), i.e.,

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

- Instance 2: a 2×3 matrix with values $\begin{bmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \end{bmatrix}$

Show the complete compilation commands and output for both test cases.

3. Modify the program to add a function `void RowSums(int m, int n, int **matrix, int *rowSums)` that computes the sum of each row and stores the results in the array `rowSums`. Execute this modified version on Instance 1 and show the output.

Solution

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Function to compute the sum of all elements in a matrix
int SumMatrix(int m, int n, int **matrix)
{
    int sum = 0;
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += matrix[i][j];
        }
    }
    return sum;
}

int main()
{
    // Test case 1: 3x4 matrix with sequential values
    int m1 = 3, n1 = 4;

    int** mat1 = new int*[m1];
    for (int i = 0; i < m1; i++)
    {
        mat1[i] = new int[n1];
    }

    int val = 1;
    for (int i = 0; i < m1; i++)
    {
```

```
32          for (int j = 0; j < n1; j++)
33          {
34              mat1[i][j] = val++;
35          }
36      }
37
38      cout << "Test 1 - Matrix " << m1 << "x" << n1 << ":" << endl;
39      for (int i = 0; i < m1; i++)
40      {
41          for (int j = 0; j < n1; j++)
42          {
43              cout << mat1[i][j] << " ";
44          }
45          cout << endl;
46      }
47      cout << "Sum: " << SumMatrix(m1, n1, mat1) << endl << endl;
48
49      for (int i = 0; i < m1; i++)
50      {
51          delete[] mat1[i];
52      }
53      delete[] mat1;
54
55      // Test case 2: 2x3 matrix
56      int m2 = 2, n2 = 3;
57      int data[2][3] = {{3, 1, 4}, {1, 5, 9}};
58
59      int** mat2 = new int*[m2];
60      for (int i = 0; i < m2; i++)
61      {
62          mat2[i] = new int[n2];
63          for (int j = 0; j < n2; j++)
64          {
65              mat2[i][j] = data[i][j];
66          }
67      }
68
69      cout << "Test 2 - Matrix " << m2 << "x" << n2 << ":" << endl;
70      for (int i = 0; i < m2; i++)
71      {
72          for (int j = 0; j < n2; j++)
73          {
74              cout << mat2[i][j] << " ";
75          }
76          cout << endl;
77      }
78      cout << "Sum: " << SumMatrix(m2, n2, mat2) << endl;
79
80      for (int i = 0; i < m2; i++)
81      {
82          delete[] mat2[i];
83      }
84      delete[] mat2;
85
86      return 0;
87  }
```

2. **Compilation and execution:**

   To compile the program:

   ```
   g++ -Wall -std=c++17 -o sum_matrix sum_matrix.cpp
   ```

   To run the program:

   ```
   ./sum_matrix
   ```

   Output:

   ```
   Test 1 - Matrix 3x4:
   1 2 3 4
   5 6 7 8
   9 10 11 12
   Sum: 78

   Test 2 - Matrix 2x3:
   3 1 4
   1 5 9
   Sum: 23
   ```

3. **Row sums function:**

```cpp
#include <iostream>
using namespace std;

// Function to compute the sum of all elements in a matrix
int SumMatrix(int m, int n, int **matrix)
{
    int sum = 0;
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += matrix[i][j];
        }
    }
    return sum;
}

// Function to compute the sum of each row
void RowSums(int m, int n, int **matrix, int *rowSums)
{
    for (int i = 0; i < m; i++)
    {
```

```
        rowSums[i] = 0;
        for (int j = 0; j < n; j++)
        {
            rowSums[i] += matrix[i][j];
        }
    }
}

int main()
{
    // Test case 1: 3x4 matrix with sequential values
    int m1 = 3, n1 = 4;

    int** mat1 = new int*[m1];
    for (int i = 0; i < m1; i++)
    {
        mat1[i] = new int[n1];
    }

    int val = 1;
    for (int i = 0; i < m1; i++)
    {
        for (int j = 0; j < n1; j++)
        {
            mat1[i][j] = val++;
        }
    }

    cout << "Matrix " << m1 << "x" << n1 << ":" << endl;
    for (int i = 0; i < m1; i++)
    {
        for (int j = 0; j < n1; j++)
        {
            cout << mat1[i][j] << " ";
        }
        cout << endl;
    }
    cout << "Total sum: " << SumMatrix(m1, n1, mat1) << endl << endl;

    int* rowSums = new int[m1];
    RowSums(m1, n1, mat1, rowSums);

    cout << "Row sums:" << endl;
    for (int i = 0; i < m1; i++)
    {
        cout << "Row " << (i + 1) << ": " << rowSums[i] << endl;
    }

    delete[] rowSums;
    for (int i = 0; i < m1; i++)
    {
        delete[] mat1[i];
    }
    delete[] mat1;

    return 0;
```

```
79 }
```

Output:

```
Matrix 3x4:
1 2 3 4
5 6 7 8
9 10 11 12
Total sum: 78

Row sums:
Row 1: 10
Row 2: 26
Row 3: 42
```

Chapter 7

Introduction to Python

In the exercises of this chapter the programs are written and executed with the conventions of the chapter «Introduction to `Python`», summarized below.

Warning

- The program of an exercise is saved in a file `<name>.py` and executed with the command

  ```
  python3 <name>.py
  ```

- The test instances are lists written in the code, for example `arr1 = [9, 12, 3, 16, 19, 11, 12]`, and are passed to the functions without their size: a function obtains the size n of a list `a` with `len(a)`.
- The elements of a list of size n are `a[0]`, `a[1]`, ..., `a[n-1]`: the entry $a[i]$ of an instance written as $a = [\dots]$ in the text corresponds to `a[i-1]` in the code, and the indices printed by the programs start from 0.
- The main instructions are written in the block `if __name__ == "__main__":`; a list is printed with `print`, which displays it in square brackets.
- The results that a function has to give back together are returned as a tuple (`return x, y`) and received with a multiple assignment (`x, y = f(a)`).
- The format `:.4f` inside an f-string displays a floating-point number with exactly 4 digits after the decimal point.

Exercise 7.1 — Introduction to `Python` ($E1$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of finding the maximum value in the array.

Questions

1. Write a complete `Python` program that finds the maximum value in a list a of n integer numbers. The program should:
   - define a function `array_max_value(a)` that implements the algorithm;
   - in the `__main__` block, create test lists and call the function;
   - print the results for each test case.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the command that executes the program and its output for all test cases.

3. Modify the function so that it returns both the maximum value and its index in the list. Execute the modified version on Instance 1 and show the output.

Solution

1. **Complete `Python` program:**

```python
def array_max_value(a):
    """Find and return the maximum value in list a."""
    max_val = a[0]  # Initialize with the first element

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]  # Update the maximum

    return max_val


if __name__ == "__main__":
    # Test case 1
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    print(f"Test 1 - List: {arr1}")
    print(f"Maximum value: {array_max_value(arr1)}")
    print()

    # Test case 2
    arr2 = [5, 2, 8, 1]
    print(f"Test 2 - List: {arr2}")
    print(f"Maximum value: {array_max_value(arr2)}")
    print()

    # Test case 3
```

```
    arr3 = [42]
    print(f"Test 3 - List: {arr3}")
    print(f"Maximum value: {array_max_value(arr3)}")
```

For Instance 3 the loop `for i in range(1, 1)` performs no iteration, and the function returns the only element `a[0]`.

2. **Execution.** If the program is saved in the file `max_value.py`, it is executed with the command

   ```
   python3 max_value.py
   ```

   Output:

   ```
   Test 1 - List: [9, 12, 3, 16, 19, 11, 12]
   Maximum value: 19

   Test 2 - List: [5, 2, 8, 1]
   Maximum value: 8

   Test 3 - List: [42]
   Maximum value: 42
   ```

3. **Modified function returning the maximum value and its index:**

```
def array_max_value_with_index(a):
    """Return (max_value, index) of the maximum element in list a."""
    max_val = a[0]
    idx = 0

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
            idx = i  # Store the index

    return max_val, idx


if __name__ == "__main__":
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    max_val, idx = array_max_value_with_index(arr1)
    print(f"Maximum value: {max_val}, at index: {idx}")
```

   Output:

   ```
   Maximum value: 19, at index: 4
   ```

For Instance 1 the function returns the tuple (19, 4): the maximum value 19 is at index 4 of the list (indices from 0), that is, in position 5 of the array $a[1..7]$ of the text. If the maximum value appears more than once, the function returns the index of its first occurrence, because `max_val` and `idx` are updated only when `a[i] > max_val` (strict inequality).

Exercise 7.2 — Introduction to `Python` ($E2$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of finding its minimum value.

Questions

1. Write a complete `Python` program that finds the minimum value in a list a of n integer numbers. The program should:
   - define a function `array_min_value(a)` that implements the algorithm;
   - in the `__main__` block, create test lists and call the function;
   - print the results for each test case.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the command that executes the program and its output for all test cases.

3. Modify the function so that it returns both the minimum value and its index in the list. Execute the modified version on Instance 1 and show the output.

Solution

1. **Complete `Python` program:**

```python
def array_min_value(a):
    """Find and return the minimum value in list a."""
    min_val = a[0]  # Initialize with the first element

    for i in range(1, len(a)):
        if a[i] < min_val:
            min_val = a[i]  # Update the minimum

```

```
    return min_val


if __name__ == "__main__":
    # Test case 1
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    print(f"Test 1 - List: {arr1}")
    print(f"Minimum value: {array_min_value(arr1)}")
    print()

    # Test case 2
    arr2 = [5, 2, 8, 1]
    print(f"Test 2 - List: {arr2}")
    print(f"Minimum value: {array_min_value(arr2)}")
    print()

    # Test case 3
    arr3 = [42]
    print(f"Test 3 - List: {arr3}")
    print(f"Minimum value: {array_min_value(arr3)}")
```

For Instance 3 the loop `for i in range(1, 1)` performs no iteration, and the function returns the only element `a[0]`.

2. **Execution.** If the program is saved in the file `min_value.py`, it is executed with the command

   ```
   python3 min_value.py
   ```

   Output:

   ```
   Test 1 - List: [9, 12, 3, 16, 19, 11, 12]
   Minimum value: 3

   Test 2 - List: [5, 2, 8, 1]
   Minimum value: 1

   Test 3 - List: [42]
   Minimum value: 42
   ```

3. **Modified function returning the minimum value and its index:**

   ```
   def array_min_value_with_index(a):
       """Return (min_value, index) of the minimum element in list a."""
       min_val = a[0]
       idx = 0

   ```

```
    for i in range(1, len(a)):
        if a[i] < min_val:
            min_val = a[i]
            idx = i  # Store the index

    return min_val, idx


if __name__ == "__main__":
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    min_val, idx = array_min_value_with_index(arr1)
    print(f"Minimum value: {min_val}, at index: {idx}")
```

Output:

```
Minimum value: 3, at index: 2
```

For Instance 1 the function returns the tuple (3, 2): the minimum value 3 is at index 2 of the list (indices from 0), that is, in position 3 of the array $a[1..7]$ of the text. If the minimum value appears more than once, the function returns the index of its first occurrence, because `min_val` and `idx` are updated only when `a[i] < min_val` (strict inequality).

Exercise 7.3 — Introduction to Python ($E3$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of computing the sum and average of all its elements.

Questions

1. Write a complete `Python` program that computes the sum and the average of the elements of a list a of n integer numbers. The program should:
   - define a function `array_sum(a)` that returns the sum of all elements;
   - define a function `array_average(a)` that returns the average of all elements;
   - in the `__main__` block, create test lists, call both functions, and print the results, with the average rounded to 4 decimal digits.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

Show the command that executes the program and its output for all test cases.

3. Combine the two functions into a single function `array_sum_and_average(a)` that returns both the sum and the average as a tuple. Execute it on Instance 1 and show the output.

Solution

1. **Complete `Python` program:**

```python
def array_sum(a):
    """Compute and return the sum of all elements in list a."""
    total = 0
    for x in a:
        total += x
    return total


def array_average(a):
    """Compute and return the average of all elements in list a."""
    return array_sum(a) / len(a)


if __name__ == "__main__":
    # Test case 1
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    print(f"Test 1 - List: {arr1}")
    print(f"Sum:     {array_sum(arr1)}")
    print(f"Average: {array_average(arr1):.4f}")
    print()

    # Test case 2
    arr2 = [5, 2, 8, 1]
    print(f"Test 2 - List: {arr2}")
    print(f"Sum:     {array_sum(arr2)}")
    print(f"Average: {array_average(arr2):.4f}")
    print()

    # Test case 3
    arr3 = [42]
    print(f"Test 3 - List: {arr3}")
    print(f"Sum:     {array_sum(arr3)}")
    print(f"Average: {array_average(arr3):.4f}")
```

The loop `for x in a` visits the elements of the list directly. The operator `/` always returns a `float`, so no type conversion is needed (in `C++` the sum must be converted to `double` before the division). The function `array_average` requires $n \geq 1$: for an empty list the division by `len(a)` $= 0$ stops the program with a `ZeroDivisionError`.

2. **Execution.** If the program is saved in the file `array_sum.py`, it is executed with the command

   ```
   python3 array_sum.py
   ```

   Output:

   ```
   Test 1 - List: [9, 12, 3, 16, 19, 11, 12]
   Sum:     82
   Average: 11.7143

   Test 2 - List: [5, 2, 8, 1]
   Sum:     16
   Average: 4.0000

   Test 3 - List: [42]
   Sum:     42
   Average: 42.0000
   ```

   For Instance 1 the exact average is $82/7 = 11.714285\ldots$, displayed as 11.7143.

3. **Combined function:**

   ```python
   def array_sum_and_average(a):
       """Return (sum, average) of list a."""
       total = 0
       for x in a:
           total += x
       avg = total / len(a)
       return total, avg


   if __name__ == "__main__":
       arr1 = [9, 12, 3, 16, 19, 11, 12]
       s, avg = array_sum_and_average(arr1)
       print(f"Sum: {s}, Average: {avg:.4f}")
   ```

   Output:

   ```
   Sum: 82, Average: 11.7143
   ```

   The combined function visits the list only once, while `array_average` of the first program calls `array_sum` and visits the list a second time.

Exercise 7.4 — Introduction to Python ($E4$)

Given an array a of n integer numbers and a value m, we consider the problem of determining whether the value m is present or not in the array.

Questions

1. Write a complete `Python` program that performs a **linear search** in a list a. The program should:
   - define a function `linear_search(a, m)` that searches for the value `m` in the list `a`;
   - return the index of the first occurrence of `m` if it is found, or `-1` if it is not found;
   - in the `__main__` block, run the test cases and print the results.

2. Execute the program on the list $a = [9, 12, 3, 16, 19, 11, 12]$ with the following values:
   - $m = 16$ (present in the list)
   - $m = 12$ (present twice: the index of the first occurrence is returned)
   - $m = 7$ (not present in the list)

   Show the command that executes the program and its output for all test cases.

3. Modify the function so that it returns the **list of all indices** where the value m appears (instead of only the first occurrence). Execute it with $m = 12$ and show the output.

Solution

1. **Complete `Python` program:**

```python
def linear_search(a, m):
    """Search for the value m in list a.
    Return the index of the first occurrence, or -1 if not found."""
    for i in range(len(a)):
        if a[i] == m:
            return i  # Found: return the index
    return -1         # Not found


def print_result(a, m):
    """Search for m in a and print the result."""
    idx = linear_search(a, m)
    if idx != -1:
```

```
        print(f"Value {m} found at index {idx}")
    else:
        print(f"Value {m} not found")


if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]
    print(f"List: {a}")
    print()

    print_result(a, 16)
    print_result(a, 12)
    print_result(a, 7)
```

The instruction `return i` terminates the function, and therefore the loop, at the first occurrence of `m`; the instruction `return -1` is reached only when the loop has visited the whole list without finding `m`. The value `-1` is used because it cannot be an index of the list. The auxiliary function `print_result` avoids writing three times the same instructions.

2. **Execution.** If the program is saved in the file `linear_search.py`, it is executed with the command

   ```
   python3 linear_search.py
   ```

   Output:

   ```
   List: [9, 12, 3, 16, 19, 11, 12]

   Value 16 found at index 3
   Value 12 found at index 1
   Value 7 not found
   ```

   The value 12 is at the indices 1 and 6, and the function returns the first one. In the array $a[1..7]$ of the text, the indices 3 and 1 correspond to the positions 4 and 2.

3. **Function returning all the occurrences:**

```
def linear_search_all(a, m):
    """Return the list of all indices where the value m appears in a.
    Return an empty list if m is not found."""
    indices = []
    for i in range(len(a)):
        if a[i] == m:
            indices.append(i)
    return indices

```

```python

if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]

    m = 12
    result = linear_search_all(a, m)
    if result:  # A non-empty list is true, an empty list is false
        print(f"Value {m} found at indices: {result}")
    else:
        print(f"Value {m} not found")
```

Output:

```
Value 12 found at indices: [1, 6]
```

Unlike `linear_search`, this function always visits the whole list, because it cannot stop at the first occurrence. The list `indices` starts empty and grows with `append`; when it is used as a condition, an empty list counts as `False`.

Exercise 7.5 — Introduction to Python ($E5$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of computing in a single pass the maximum, minimum, sum and average of all elements in the array.

Questions

1. Write a complete `Python` program that computes in a **single pass** the four statistics (maximum, minimum, sum, average) of a list a of n integer numbers. The program should:
   - define a function `array_stats(a)` that computes the four values with a single loop;
   - return the four values as a tuple `(max_val, min_val, total, avg)`;
   - in the `__main__` block, run the test cases and print the results.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the command that executes the program and its output for all test cases.

3. Modify the function so that it also returns the indices of the maximum and of the minimum element. Execute it on Instance 1 and show the output.

Solution

1. **Complete `Python` program:**

```python
def array_stats(a):
    """Compute max, min, sum, and average in a single pass.
    Return (max_val, min_val, total, avg)."""
    max_val = a[0]
    min_val = a[0]
    total = a[0]

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
        if a[i] < min_val:
            min_val = a[i]
        total += a[i]

    avg = total / len(a)
    return max_val, min_val, total, avg


def print_stats(k, a):
    """Print the statistics of the test case k."""
    max_v, min_v, s, avg = array_stats(a)
    print(f"Test {k} - List: {a}")
    print(f"  Maximum: {max_v}")
    print(f"  Minimum: {min_v}")
    print(f"  Sum:     {s}")
    print(f"  Average: {avg:.4f}")


if __name__ == "__main__":
    print_stats(1, [9, 12, 3, 16, 19, 11, 12])
    print()
    print_stats(2, [5, 2, 8, 1])
    print()
    print_stats(3, [42])
```

The three variables are initialized with the first element, and the loop visits the other $n - 1$ elements once: at each iteration the current element is compared with the maximum and with the minimum, and it is added to the sum.

2. **Execution.** If the program is saved in the file `array_stats.py`, it is executed with the command

```
python3 array_stats.py
```

Output:

```
Test 1 - List: [9, 12, 3, 16, 19, 11, 12]
  Maximum: 19
  Minimum: 3
  Sum:     82
  Average: 11.7143

Test 2 - List: [5, 2, 8, 1]
  Maximum: 8
  Minimum: 1
  Sum:     16
  Average: 4.0000

Test 3 - List: [42]
  Maximum: 42
  Minimum: 42
  Sum:     42
  Average: 42.0000
```

3. **Extended function also returning the indices:**

```python
def array_stats_with_indices(a):
    """Compute all the statistics, including the positions.
    Return (max_val, max_idx, min_val, min_idx, total, avg)."""
    max_val = a[0]
    max_idx = 0
    min_val = a[0]
    min_idx = 0
    total = a[0]

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
            max_idx = i
        if a[i] < min_val:
            min_val = a[i]
            min_idx = i
        total += a[i]

    avg = total / len(a)
    return max_val, max_idx, min_val, min_idx, total, avg


if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]
    max_v, max_i, min_v, min_i, s, avg = array_stats_with_indices(a)
    print(f"List:    {a}")
    print(f"Max:     {max_v} at index {max_i}")
    print(f"Min:     {min_v} at index {min_i}")
    print(f"Sum:     {s}")
    print(f"Average: {avg:.4f}")
```

Output:

```
List:    [9, 12, 3, 16, 19, 11, 12]
Max:     19 at index 4
Min:     3 at index 2
Sum:     82
Average: 11.7143
```

The maximum 19 is at index 4 and the minimum 3 at index 2 (indices from 0), that is, in the positions 5 and 3 of the array $a[1..7]$ of the text.

Chapter 8

Introduction to Java

In the exercises of this chapter the programs are written, compiled and executed with the conventions of the chapter «Introduction to `Java`», summarized below.

Warning

- The program of an exercise is a class `Main`, saved in the file `Main.java` (each exercise in its own folder), compiled and executed with the commands

```
javac Main.java
java Main
```

- The methods are `static` methods of the class `Main`. The test instances are arrays written in the code, for example `int[] arr1 = {9, 12, 3, 16, 19, 11, 12};`, and are passed to the methods without their size: a method obtains the size n of an array `a` with `a.length`.
- The elements of an array of size n are `a[0]`, `a[1]`, ..., `a[n-1]`: the entry $a[i]$ of an instance written as $a = [\dots]$ in the text corresponds to `a[i-1]` in the code, and the indices printed by the programs start from 0.
- A method returns at most one value: the results that a method has to give back together are returned in an array (`int[]` or `double[]`).
- When an array is printed with a loop, every value is followed by a space, so the printed lines end with a space.
- The format `%.4f` of `printf` displays a floating-point number with exactly 4 digits after the decimal point. The outputs are the ones of a computer with English language settings: with Italian settings `printf` writes the decimal comma (`11,7143`).

Exercise 8.1 — Introduction to `Java` ($E1$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of finding the maximum value in the array.

Questions

1. Write a complete `Java` program that finds the maximum value in an array a of n integer numbers. The program should:
   - define a static method `int arrayMaxValue(int[] a)` inside the class `Main`;
   - in the `main` method, create test arrays and call the method;
   - print the results for each test case.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the compilation and execution commands and the output for all test cases.

3. Modify the method so that it returns both the maximum value and its index. Since a `Java` method returns only one value, return an array `int[]` of two elements: `{maxValue, maxIndex}`. Execute the modified version on Instance 1 and show the output.

Solution

1. **Complete `Java` program:**

```java
public class Main {

    static int arrayMaxValue(int[] a) {
        int max = a[0]; // Initialize with the first element

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) {
                max = a[i]; // Update the maximum
            }
        }
        return max;
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        System.out.print("Test 1 - Array: ");
        for (int x : arr1) System.out.print(x + " ");
        System.out.println();
        System.out.println("Maximum value: " + arrayMaxValue(arr1));
        System.out.println();

        int[] arr2 = {5, 2, 8, 1};
        System.out.print("Test 2 - Array: ");
```

```
        for (int x : arr2) System.out.print(x + " ");
        System.out.println();
        System.out.println("Maximum value: " + arrayMaxValue(arr2));
        System.out.println();

        int[] arr3 = {42};
        System.out.print("Test 3 - Array: ");
        for (int x : arr3) System.out.print(x + " ");
        System.out.println();
        System.out.println("Maximum value: " + arrayMaxValue(arr3));
    }
}
```

The arrays are printed with the enhanced `for` loop, which visits their elements directly. For Instance 3 the condition `1 < a.length` is false from the beginning, the loop performs no iteration, and the method returns the only element `a[0]`.

2. **Compilation and execution:**

```
javac Main.java
java Main
```

Output:

```
Test 1 - Array: 9 12 3 16 19 11 12
Maximum value: 19

Test 2 - Array: 5 2 8 1
Maximum value: 8

Test 3 - Array: 42
Maximum value: 42
```

3. **Modified method returning the maximum value and its index:**

```
public class Main {

    // Returns int[]{maxValue, maxIndex}
    static int[] arrayMaxValueWithIndex(int[] a) {
        int max = a[0];
        int idx = 0;

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) {
                max = a[i];
                idx = i; // Store the index
            }
        }
        return new int[]{max, idx};
```

```java
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        int[] result = arrayMaxValueWithIndex(arr1);
        System.out.println("Maximum value: " + result[0]
                           + ", at index: " + result[1]);
    }
}
```

Output:

```
Maximum value: 19, at index: 4
```

For Instance 1 the method returns the array $\{19,\ 4\}$: the maximum value 19 is at index 4 (indices from 0), that is, in position 5 of the array $a[1..7]$ of the text. If the maximum value appears more than once, the method returns the index of its first occurrence, because `max` and `idx` are updated only when `a[i] > max` (strict inequality).

Exercise 8.2 — Introduction to Java ($E2$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of finding its minimum value.

Questions

1. Write a complete `Java` program that finds the minimum value in an array a of n integer numbers. The program should:
   - define a static method `int arrayMinValue(int[] a)` inside the class `Main`;
   - in the `main` method, create test arrays and call the method;
   - print the results for each test case.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the compilation and execution commands and the output for all test cases.

3. Modify the method so that it returns both the minimum value and its index. Since a `Java` method returns only one value, return an array `int[]` of two elements: `{minValue, minIndex}`. Execute the modified version on Instance 1 and show the output.

Solution

1. **Complete Java program:**

```java
public class Main {

    static int arrayMinValue(int[] a) {
        int min = a[0]; // Initialize with the first element

        for (int i = 1; i < a.length; i++) {
            if (a[i] < min) {
                min = a[i]; // Update the minimum
            }
        }
        return min;
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        System.out.print("Test 1 - Array: ");
        for (int x : arr1) System.out.print(x + " ");
        System.out.println();
        System.out.println("Minimum value: " + arrayMinValue(arr1));
        System.out.println();

        int[] arr2 = {5, 2, 8, 1};
        System.out.print("Test 2 - Array: ");
        for (int x : arr2) System.out.print(x + " ");
        System.out.println();
        System.out.println("Minimum value: " + arrayMinValue(arr2));
        System.out.println();

        int[] arr3 = {42};
        System.out.print("Test 3 - Array: ");
        for (int x : arr3) System.out.print(x + " ");
        System.out.println();
        System.out.println("Minimum value: " + arrayMinValue(arr3));
    }
}
```

The arrays are printed with the enhanced `for` loop, which visits their elements directly. For Instance 3 the condition `1 < a.length` is false from the beginning, the loop performs no iteration, and the method returns the only element `a[0]`.

2. **Compilation and execution:**

```
javac Main.java
java Main
```

Output:

```
Test 1 - Array: 9 12 3 16 19 11 12
```

```
Minimum value: 3

Test 2 - Array: 5 2 8 1
Minimum value: 1

Test 3 - Array: 42
Minimum value: 42
```

3. **Modified method returning the minimum value and its index:**

```java
public class Main {

    // Returns int[]{minValue, minIndex}
    static int[] arrayMinValueWithIndex(int[] a) {
        int min = a[0];
        int idx = 0;

        for (int i = 1; i < a.length; i++) {
            if (a[i] < min) {
                min = a[i];
                idx = i; // Store the index
            }
        }
        return new int[]{min, idx};
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        int[] result = arrayMinValueWithIndex(arr1);
        System.out.println("Minimum value: " + result[0]
                           + ", at index: " + result[1]);
    }
}
```

Output:

```
Minimum value: 3, at index: 2
```

For Instance 1 the method returns the array $\{3,\ 2\}$: the minimum value 3 is at index 2 (indices from 0), that is, in position 3 of the array $a[1..7]$ of the text. If the minimum value appears more than once, the method returns the index of its first occurrence, because `min` and `idx` are updated only when `a[i] < min` (strict inequality).

Exercise 8.3 — Introduction to Java ($E3$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of computing the sum and average of all its elements.

Questions

1. Write a complete `Java` program that computes the sum and the average of the elements of an array a of n integer numbers. The program should:
   - define a static method `int arraySum(int[] a)` that returns the sum;
   - define a static method `double arrayAverage(int[] a)` that returns the average;
   - in the `main` method, create test arrays, call both methods, and print the results, with the average rounded to 4 decimal digits.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the compilation and execution commands and the output for all test cases.

3. Write a single method `arraySumAndAverage(int[] a)` that returns both the sum and the average. Since a `Java` method returns only one value, return the result as an array `double[]` of two elements: `{sum, average}`. Execute it on Instance 1 and show the output.

Solution

1. **Complete `Java` program:**

```java
public class Main {

    static int arraySum(int[] a) {
        int total = 0;
        for (int i = 0; i < a.length; i++) {
            total += a[i];
        }
        return total;
    }

    static double arrayAverage(int[] a) {
        return (double) arraySum(a) / a.length;
    }

    static void printTest(int k, int[] a) {
        System.out.print("Test " + k + " - Array: ");
        for (int x : a) System.out.print(x + " ");
        System.out.println();
        System.out.println("Sum:     " + arraySum(a));
        System.out.printf("Average: %.4f%n", arrayAverage(a));
    }
```

```
22
23      public static void main(String[] args) {
24          int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
25          printTest(1, arr1);
26          System.out.println();
27
28          int[] arr2 = {5, 2, 8, 1};
29          printTest(2, arr2);
30          System.out.println();
31
32          int[] arr3 = {42};
33          printTest(3, arr3);
34      }
35  }
```

The conversion `(double)` is necessary: without it, `arraySum(a) / a.length` would be the integer division, and for Instance 1 the average would be $82/7 = 11$. The method `printTest` prints the array and the two results, and avoids writing three times the same instructions.

2. **Compilation and execution:**

```
javac Main.java
java Main
```

Output:

```
Test 1 - Array: 9 12 3 16 19 11 12
Sum:     82
Average: 11.7143

Test 2 - Array: 5 2 8 1
Sum:     16
Average: 4.0000

Test 3 - Array: 42
Sum:     42
Average: 42.0000
```

For Instance 1 the exact average is $82/7 = 11.714285\ldots$, displayed as 11.7143.

3. **Combined method:**

```
1  public class Main {
2
3      // Returns double[]{sum, average}
4      static double[] arraySumAndAverage(int[] a) {
5          int total = 0;
6          for (int x : a) total += x;
```

```
        double avg = (double) total / a.length;
        return new double[]{total, avg};
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        double[] result = arraySumAndAverage(arr1);
        System.out.printf("Sum: %.0f, Average: %.4f%n",
                          result[0], result[1]);
    }
}
```

Output:

```
Sum: 82, Average: 11.7143
```

The integer `total` is converted automatically into a `double` when it is stored in the array `double[]`; for this reason it is printed with the format `%.0f` (no decimal digits) instead of `%d`, which accepts only integers.

Exercise 8.4 — Introduction to Java ($E4$)

Given an array a of n integer numbers and a value m, we consider the problem of determining whether the value m is present or not in the array.

Questions

1. Write a complete `Java` program that performs a **linear search** in an array a. The program should:
   - define a static method `int linearSearch(int[] a, int m)` that searches for the value `m` in the array `a`;
   - return the index of the first occurrence of `m` if it is found, or `-1` if it is not found;
   - in the `main` method, run the test cases and print the results.

2. Execute the program on the array $a = [9, 12, 3, 16, 19, 11, 12]$ with the following values:
   - $m = 16$ (present in the array)
   - $m = 12$ (present twice: the index of the first occurrence is returned)
   - $m = 7$ (not present in the array)

   Show the compilation and execution commands and the output for all test cases.

3. Modify the method so that it returns an array `int[]` containing **all the indices** where the value m appears (instead of only the first occurrence). Use an `ArrayList<Integer>` inside the method and convert it into an `int[]` before returning. Execute it with $m = 12$ and show the output.

Solution

1. **Complete Java program:**

```java
public class Main {

    static int linearSearch(int[] a, int m) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == m) {
                return i; // Found: return the index
            }
        }
        return -1; // Not found
    }

    static void printResult(int[] a, int m) {
        int idx = linearSearch(a, m);
        if (idx != -1) System.out.println("Value " + m + " found at index " + idx);
        else           System.out.println("Value " + m + " not found");
    }

    public static void main(String[] args) {
        int[] a = {9, 12, 3, 16, 19, 11, 12};
        System.out.print("Array: ");
        for (int x : a) System.out.print(x + " ");
        System.out.println();
        System.out.println();

        printResult(a, 16);
        printResult(a, 12);
        printResult(a, 7);
    }
}
```

The instruction `return i` terminates the method, and therefore the loop, at the first occurrence of `m`; the instruction `return -1` is reached only when the loop has visited the whole array without finding `m`. The value `-1` is used because it cannot be an index of the array.

2. **Compilation and execution:**

```
javac Main.java
java Main
```

Output:

```
Array: 9 12 3 16 19 11 12

Value 16 found at index 3
Value 12 found at index 1
Value 7 not found
```

The value 12 is at the indices 1 and 6, and the method returns the first one. In the array $a[1..7]$ of the text, the indices 3 and 1 correspond to the positions 4 and 2.

3. **Method returning all the occurrences:**

```java
import java.util.ArrayList;

public class Main {

    static int[] linearSearchAll(int[] a, int m) {
        ArrayList<Integer> indices = new ArrayList<>();
        for (int i = 0; i < a.length; i++) {
            if (a[i] == m) {
                indices.add(i);
            }
        }
        // Conversion of the ArrayList<Integer> into an int[]
        int[] result = new int[indices.size()];
        for (int i = 0; i < indices.size(); i++) {
            result[i] = indices.get(i);
        }
        return result;
    }

    public static void main(String[] args) {
        int[] a = {9, 12, 3, 16, 19, 11, 12};
        int m = 12;
        int[] occurrences = linearSearchAll(a, m);

        if (occurrences.length > 0) {
            System.out.print("Value " + m + " found at indices: ");
            for (int idx : occurrences) System.out.print(idx + " ");
            System.out.println();
        } else {
            System.out.println("Value " + m + " not found");
        }
    }
}
```

Output:

```
Value 12 found at indices: 1 6
```

The number of occurrences is not known in advance, so the indices are first collected in an `ArrayList`, which grows with `add`; then an array of the right size `indices.size()` is allocated and filled. Unlike `linearSearch`, this method always visits the whole array. If `m` is not present, the method returns an array of length 0.

Exercise 8.5 — Introduction to Java ($E5$)

Given an array a of $n \geq 1$ integer numbers, we consider the problem of computing in a single pass the maximum, minimum, sum and average of all elements in the array.

Questions

1. Write a complete `Java` program that computes in a **single pass** the four statistics (maximum, minimum, sum, average) of an array a of n integer numbers. The program should:
   - define a static method `arrayStats(int[] a)` that computes the four values with a single loop;
   - return the results as an array `double[]` of four elements: `{max, min, sum, average}`;
   - in the `main` method, run the test cases and print the results.

2. Execute the program on the following instances:
   - Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$
   - Instance 2: $a = [5, 2, 8, 1]$ with $n = 4$
   - Instance 3: $a = [42]$ with $n = 1$

   Show the compilation and execution commands and the output for all test cases.

3. Modify the method so that it also returns the indices of the maximum and of the minimum element, as an array `double[]` of six elements: `{max, maxIdx, min, minIdx, sum, average}`. Execute it on Instance 1 and show the output.

Solution

1. **Complete `Java` program:**

```java
public class Main {

    // Returns double[]{max, min, sum, average}
    static double[] arrayStats(int[] a) {
        int max = a[0];
        int min = a[0];
        int total = a[0];

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) max = a[i];
            if (a[i] < min) min = a[i];
            total += a[i];
```

```
        }

        double avg = (double) total / a.length;
        return new double[]{max, min, total, avg};
    }

    static void printStats(int k, int[] a) {
        double[] s = arrayStats(a);
        System.out.print("Test " + k + " - Array: ");
        for (int x : a) System.out.print(x + " ");
        System.out.println();
        System.out.println("  Maximum: " + (int) s[0]);
        System.out.println("  Minimum: " + (int) s[1]);
        System.out.println("  Sum:     " + (int) s[2]);
        System.out.printf("  Average: %.4f%n", s[3]);
    }

    public static void main(String[] args) {
        printStats(1, new int[]{9, 12, 3, 16, 19, 11, 12});
        System.out.println();
        printStats(2, new int[]{5, 2, 8, 1});
        System.out.println();
        printStats(3, new int[]{42});
    }
}
```

The three variables are initialized with the first element, and the loop visits the other $n-1$ elements once. The four results have different data types, so they are all stored as `double` in the returned array; the integer results are converted back with `(int)` before printing, so that they are displayed without the decimal part (`19` instead of `19.0`). The expression `new int[]{...}` creates an array directly in the call.

2. **Compilation and execution:**

```
javac Main.java
java Main
```

Output:

```
Test 1 - Array: 9 12 3 16 19 11 12
  Maximum: 19
  Minimum: 3
  Sum:     82
  Average: 11.7143

Test 2 - Array: 5 2 8 1
  Maximum: 8
  Minimum: 1
  Sum:     16
  Average: 4.0000
```

```
Test 3 - Array: 42
  Maximum: 42
  Minimum: 42
  Sum:     42
  Average: 42.0000
```

3. **Extended method also returning the indices:**

```java
public class Main {

    // Returns double[]{max, maxIdx, min, minIdx, sum, average}
    static double[] arrayStatsWithIndices(int[] a) {
        int max = a[0], maxIdx = 0;
        int min = a[0], minIdx = 0;
        int total = a[0];

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) { max = a[i]; maxIdx = i; }
            if (a[i] < min) { min = a[i]; minIdx = i; }
            total += a[i];
        }

        double avg = (double) total / a.length;
        return new double[]{max, maxIdx, min, minIdx, total, avg};
    }

    public static void main(String[] args) {
        int[] a = {9, 12, 3, 16, 19, 11, 12};
        double[] r = arrayStatsWithIndices(a);
        System.out.println("Max: " + (int) r[0] + " at index " + (int) r[1]);
        System.out.println("Min: " + (int) r[2] + " at index " + (int) r[3]);
        System.out.println("Sum: " + (int) r[4]);
        System.out.printf("Avg: %.4f%n", r[5]);
    }
}
```

Output:

```
Max: 19 at index 4
Min: 3 at index 2
Sum: 82
Avg: 11.7143
```

The maximum 19 is at index 4 and the minimum 3 at index 2 (indices from 0), that is, in the positions 5 and 3 of the array $a[1..7]$ of the text.